SQL Database and Stored Procedure Setup

This SQL script creates the **ClassDB** database, defines the **Class** and **Student** tables, and adds stored procedures for creating, reading, updating, and deleting records.

Database Setup

```
CREATE DATABASE ClassDB;
USE ClassDB;
```

Table Creation Procedures

The following stored procedures create the main database tables. The **Class** table stores class names, and the **Student** table stores student details with a foreign key relationship to **Class**.

```
CREATE PROCEDURE SP_CreateClassTable
AS
BEGIN
    CREATE TABLE Class (
        ClassId INT IDENTITY(1,1) PRIMARY KEY,
        ClassName VARCHAR(50) NOT NULL
    );
END;

EXEC SP_CreateClassTable;
```

```
CREATE PROCEDURE SP_CreateStudentTable
AS
BEGIN
    CREATE TABLE Student (
        StudentID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50) NOT NULL,
        ClassID INT NOT NULL,
        CONSTRAINT Fk_StudentClass FOREIGN KEY (ClassID) REFERENCES
Class(ClassID)
    );
END;

EXEC SP_CreateStudentTable;
```

Insert Procedures

These procedures add new class and student records to the database.

```
CREATE PROCEDURE SP_AddClass
    @ClassName VARCHAR(50)
AS
BEGIN
    INSERT INTO Class (ClassName)
    VALUES (@ClassName);
END;

EXEC SP_AddClass @ClassName = 'HR MANAGEMENT';
```

```
CREATE PROCEDURE SP_AddStudent
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @ClassID INT
AS
BEGIN
    INSERT INTO Student (FirstName, LastName, Email, ClassID)
    VALUES (@FirstName, @LastName, @Email, @ClassID);
END;

EXEC SP_AddStudent
    @FirstName = 'Sriven',
    @LastName = 'Bommakanti',
    @Email = 'Sriven@gmail.com',
    @ClassID = 1;
```

Read Procedures

These procedures retrieve either a single record by ID or all records from the related table.

```
ALTER PROCEDURE SP_GetClassById
    @ClassId INT
AS
BEGIN
    SELECT * FROM Class WHERE ClassId = @ClassId;
END;
```

```
EXEC SP_GetClassById @ClassId = 1;

CREATE PROCEDURE SP_GetStudentById
    @StudentId INT
AS
BEGIN
    SELECT * FROM Student WHERE StudentId = @StudentId;
END;

EXEC SP_GetStudentById @StudentId = 1;

CREATE PROCEDURE SP_GetAllClasses
AS
BEGIN
    SELECT * FROM Class;
END;

EXEC SP_GetAllClasses;

CREATE PROCEDURE SP_GetAllStudents
AS
BEGIN
    SELECT * FROM Student;
END;

EXEC SP_GetAllStudents;
```

Update Procedures

These procedures update existing class and student records based on their IDs.

```
CREATE PROCEDURE SP_UpdateClassById
    @ClassId INT,
    @ClassName VARCHAR(50)
AS
BEGIN
    UPDATE Class
    SET ClassName = @ClassName
    WHERE ClassId = @ClassId;
END;
```

```
EXEC SP_UpdateClassById
    @ClassId = 1,
    @ClassName = 'PR MANAGEMENT';

CREATE PROCEDURE SP_UpdateStudentById
    @StudentId INT,
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50)
AS
BEGIN
    UPDATE Student
    SET FirstName = @FirstName,
        LastName = @LastName,
        Email = @Email
    WHERE StudentId = @StudentId;
END;

EXEC SP_UpdateStudentById
    @StudentId = 1,
    @FirstName = 'abhi',
    @LastName = 'Ram',
    @Email = 'sriven@gmail.com';
```

Delete Procedures

These procedures delete class and student records by ID.

```
CREATE PROCEDURE SP_DeleteClassById
    @ClassId INT
AS
BEGIN
    DELETE FROM Class WHERE ClassId = @ClassId;
END;

EXEC SP_DeleteClassById @ClassId = 1;

CREATE PROCEDURE SP_DeleteStudentById
    @StudentId INT
AS
BEGIN
    DELETE FROM Student WHERE StudentId = @StudentId;
```

```
END;

EXEC SP_DeleteStudentById @StudentId = 1;
```

Employee Management API Controller

The following controller provides API endpoints to retrieve, add, update, and delete employee records using the application database context.

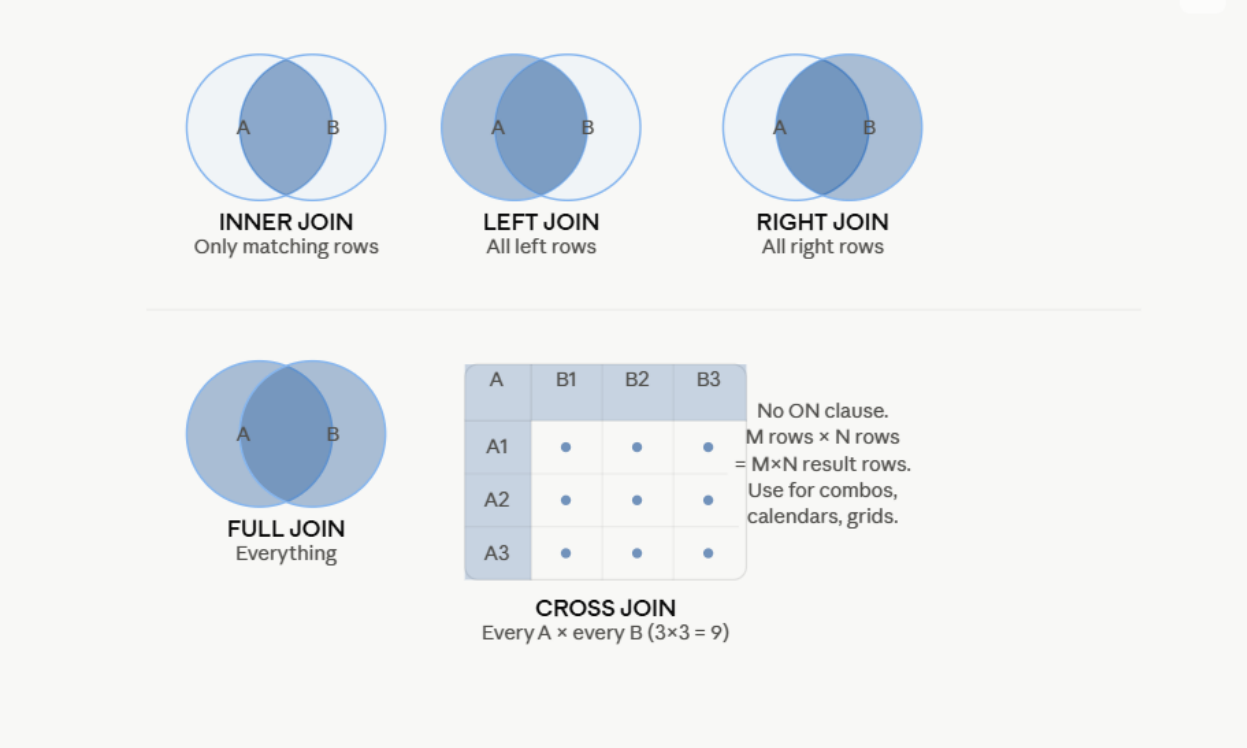
```
using EmployeeManagementAPI.Data;
using EmployeeManagementAPI.Models.Entities;
using Microsoft.AspNetCore.Mvc;
using System.Reflection.Metadata.Ecma335;

namespace EmployeeManagementAPI.Controllers
{
    [Route("api/[Controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        private readonly ApplicationDbContext _dbcontext;

        public EmployeeController(ApplicationDbContext dbcontext)
        {
            this._dbcontext = dbcontext;
        }

        [HttpGet]
        public IActionResult GetAllEmployees()
        {
            var employees = _dbcontext.Employees.ToList();
            return Ok(employees);
        }

        [HttpGet]
        [Route("{id:Guid}")]
        public IActionResult GetEmployeeById(Guid id)
        {
            var employees = _dbcontext.Employees.Find(id);

            if (employees is null)
```

```
            {
                return NotFound();
            }

            return Ok(employees);
        }

        [HttpDelete]
        [Route("{id:Guid}")]
        public IActionResult DeleteEmployeeById(Guid id)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            _dbcontext.Employees.Remove(employee);
            _dbcontext.SaveChanges();

            return Ok(employee);
        }

        [HttpPost]
        public IActionResult AddEmployee(AddEmployeeDto
addEmployeeDto)
        {
            var employee = new Employee()
            {
                FirstName = addEmployeeDto.FirstName,
                LastName = addEmployeeDto.LastName,
                Email = addEmployeeDto.Email
            };

            _dbcontext.Employees.Add(employee);
            _dbcontext.SaveChanges();
            return Ok(employee);
        }
```

```
        [HttpPut]
        [Route("{id:Guid}")]
        public IActionResult UpdateEmployeeById(Guid id,
UpdateEmployeeDto updateEmployeeDto)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            employee.FirstName = updateEmployeeDto.FirstName;
            employee.LastName = updateEmployeeDto.LastName;
            employee.Email = updateEmployeeDto.Email;

            _dbcontext.SaveChanges();
            return Ok(employee);
        }
    }
}
```

SQL Join Types

SQL joins are used to combine records from two tables based on a related column. In this example, the **Class** and **Student** tables are joined using the shared **ClassId** field.



1. INNER JOIN

An **INNER JOIN** returns only the records that have matching values in both tables.

```
SELECT *
FROM Class AS c
INNER JOIN Student AS s
ON c.ClassId = s.ClassId;
```

2. LEFT JOIN

A **LEFT JOIN** returns all records from the left table, along with matching records from the right table when available.

```
SELECT *
FROM Class AS c
LEFT JOIN Student AS s
ON c.ClassId = s.ClassId;
```

3. RIGHT JOIN

A **RIGHT JOIN** returns all records from the right table, along with matching records from the left table when available.

```
SELECT *
FROM Class AS c
RIGHT JOIN Student AS s
ON c.ClassId = s.ClassId;
```

4. FULL OUTER JOIN

A **FULL OUTER JOIN** returns all records from both tables. Matching records are combined, and unmatched records from either table are still included.

```
SELECT *
FROM Class AS c
FULL OUTER JOIN Student AS s
ON c.ClassId = s.ClassId;
```

5. CROSS JOIN

A **CROSS JOIN** returns every possible combination of records from both tables.

```
SELECT *
FROM Class AS c
CROSS JOIN Student AS s;
```

Employee Department Management Models

These entity models define the relationship between employees and departments in the Employee Department Management API. Each employee belongs to one department, while each department can contain multiple employees.

Employee Entity

The **Employee** entity stores employee details and uses **DepartmentId** to connect each employee to a department.

```
namespace EmployeeDepartmentManagementAPI.Models.Entities
{
    public class Employee
    {
        public Guid EmployeeId { get; set; }
```

```
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid DepartmentId { get; set; }
        public Department? Department { get; set; }
    }
}
```

Department Entity

The **Department** entity stores department information and includes a collection of employees assigned to that department.

```
namespace EmployeeDepartmentManagementAPI.Models.Entities
{
    public class Department
    {
        public Guid DepartmentId { get; set; }
        public string? DepartmentName { get; set; }
        public ICollection<Employee>? Employees { get; set; }
    }
}
```

Application Database Context

The **ApplicationDbContext** class connects the Employee Department Management API to the database. It inherits from **DbContext** and defines database tables for employees and departments through **DbSet** properties.

```
using EmployeeDepartmentManagementAPI.Models.Entities;
using Microsoft.EntityFrameworkCore;

namespace EmployeeDepartmentManagementAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
```

```
        public DbSet<Employee> Employees { get; set; }
        public DbSet<Department> Departments { get; set; }
    }
}
```

Application Configuration and Startup

This section configures the Employee Department Management API by defining the database connection in **appsettings.json** and registering the database context in **Program.cs**.

appsettings.json

The **appsettings.json** file stores application settings, including logging behavior, allowed hosts, and the SQL Server connection string.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=EmployeeDB;Trusted_Connection=true;TrustServerCerti
ficate=true;"
  }
}
```

Program.cs

The **Program.cs** file registers services such as controllers, Swagger, and the Entity Framework database context. It also configures the middleware pipeline used when the application runs.

```
using EmployeeDepartmentManagementAPI.Data;
using Microsoft.EntityFrameworkCore;
```

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("De
faultConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Departments API Controller

The **DepartmentsController** provides API endpoints for managing department records. It supports retrieving all departments, finding a department by ID, adding a new department, updating an existing department, and deleting a department.

```
using EmployeeDepartmentManagementAPI.Data;
using EmployeeDepartmentManagementAPI.Models;
using EmployeeDepartmentManagementAPI.Models.Entities;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeDepartmentManagementAPI.Controllers
{
```

```
[Route("api/[controller]")]
[ApiController]
public class DepartmentsController : ControllerBase
{
    private readonly ApplicationDbContext _dbcontext;

    public DepartmentsController(ApplicationDbContext dbcontext)
    {
        this._dbcontext = dbcontext;
    }

    [HttpGet]
    public IActionResult GetAllDepartments()
    {
        var departments = _dbcontext.Departments.ToList();
        return Ok(departments);
    }

    [HttpGet("{id:guid}")]
    public IActionResult GetDepartmentById(Guid id)
    {
        var department = _dbcontext.Departments.Find(id);

        if (department is null)
        {
            return NotFound();
        }

        return Ok(department);
    }

    [HttpDelete("{id:guid}")]
    public IActionResult DeleteDepartmentById(Guid id)
    {
        var department = _dbcontext.Departments.Find(id);

        if (department is null)
        {
            return NotFound();
```

```
            }

            _dbcontext.Departments.Remove(department);
            _dbcontext.SaveChanges();
            return Ok(department);
        }

        [HttpPost]
        public IActionResult AddDepartment(AddDepartmentDto
addDepartmentDto)
        {
            var department = new Department
            {
                DepartmentName = addDepartmentDto.DepartmentName
            };

            _dbcontext.Departments.Add(department);
            _dbcontext.SaveChanges();

            return Ok(department);
        }

        [HttpPut("{id:guid}")]
        public IActionResult UpdateDepartmentById(Guid id,
UpdateDepartmentDto updateDepartmentDto)
        {
            var department = _dbcontext.Departments.Find(id);

            if (department is null)
            {
                return NotFound();
            }

            department.DepartmentName =
updateDepartmentDto.DepartmentName;
            _dbcontext.SaveChanges();

            return Ok(department);
        }
```

```
    }
}
```

Employees API Controller

The **EmployeesController** provides API endpoints for managing employee records. It supports retrieving employees, finding an employee by ID, adding a new employee, updating employee details, and deleting an employee from the database.

```
using EmployeeDepartmentManagementAPI.Data;
using EmployeeDepartmentManagementAPI.Models;
using EmployeeDepartmentManagementAPI.Models.Entities;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeDepartmentManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly ApplicationDbContext _dbcontext;

        public EmployeesController(ApplicationDbContext dbcontext)
        {
            _dbcontext = dbcontext;
        }

        [HttpGet]
        public IActionResult GetAllEmployees()
        {
            var employees = _dbcontext.Employees.ToList();
            return Ok(employees);
        }

        [HttpGet("{id:guid}")]
        public IActionResult GetEmployeeById(Guid id)
        {
            var employee = _dbcontext.Employees.Find(id);
```

```
            if (employee is null)
            {
                return NotFound();
            }

            return Ok(employee);
        }

        [HttpDelete("{id:guid}")]
        public IActionResult DeleteEmployeeById(Guid id)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            _dbcontext.Employees.Remove(employee);
            _dbcontext.SaveChanges();

            return Ok(employee);
        }

        [HttpPost]
        public IActionResult AddEmployee(AddEmployeeDto
addEmployeeDto)
        {
            var employee = new Employee
            {
                FirstName = addEmployeeDto.FirstName,
                LastName = addEmployeeDto.LastName,
                Email = addEmployeeDto.Email,
                DepartmentId = addEmployeeDto.DepartmentId
            };

            _dbcontext.Employees.Add(employee);
            _dbcontext.SaveChanges();
```

```
            return Ok(employee);
        }

        [HttpPut("{id:guid}")]
        public IActionResult UpdateEmployeeById(Guid id,
UpdateEmployeeDto updateEmployeeDto)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            employee.FirstName = updateEmployeeDto.FirstName;
            employee.LastName = updateEmployeeDto.LastName;
            employee.Email = updateEmployeeDto.Email;
            employee.DepartmentId = updateEmployeeDto.DepartmentId;

            _dbcontext.SaveChanges();
            return Ok(employee);
        }
    }
}
```